>_ MCMP113 · Week 8

Midterm Review & Practice Exam

Cheat sheet · review questions · practice midterm

Asst. Prof. Hasan Oğuz, PhD
Introduction to Basic Algorithms and Programming · Istanbul Okan University
Vocational School · 2026–2027 Fall

How to use these notes

Week 8 is the midterm week, so these notes contain no new material. They have three parts. **(1) Review**: a two-page cheat sheet (Section 2) and the twelve practice questions of the lecture with worked answers (Section 3). **(2) Practice Midterm Exam** (Section 5): 90 minutes, 100 points, seven questions of the same types as the real exam. Take it *once*, closed book, on paper, with a timer. **(3) Problem set** (Section 6): twelve extra problems with difficulty stars. Every program mentioned here is in the Week 8 code folder (`08_01` ... `08_14`) and has been compiled and run; every output shown is real output. Run `08_01_PredictChecker` only *after* you have answered the predict-the-output questions yourself.

Contents

1 The midterm exam **2**
1.1 Learning outcomes checked by the midterm . . . 2
1.2 Logistics . . . 2
1.3 Question types and how to answer them . . . 3

2 Cheat sheet **4**
2.1 C# syntax summary . . . 4
2.2 Data types and operators . . . 5
2.3 Pseudocode conventions and flowchart symbols . . . 5
2.4 Loop patterns . . . 6

3 Review questions from the lecture **6**
3.1 Q1–Q4: multiple choice . . . 6
3.2 Q5: operators . . . 6
3.3 Q6: trace table . . . 7
3.4 Q7: selection . . . 7
3.5 Q8: nested loops . . . 8
3.6 Q9: `break`, `continue`, `do-while` . . . 8
3.7 Q10: find the bugs . . . 8
3.8 Q11: draw the flowchart . . . 10
3.9 Q12: write the program . . . 11

4 Common mistakes **12**

5 Practice Midterm Exam **13**

Question 1: Multiple choice 13

Question 2: Predict the output 14

Question 3: Trace table 15

Question 4: Flowchart 15

Question 5: Find and fix the bugs 15

Question 6: Write a program (selection) 16

Question 7: Write a program (loops) 16

6 Additional problem set **16**

7 Summary **17**

References and further reading **18**

>_ 1 The midterm exam

1.1 Learning outcomes checked by the midterm

After Weeks 1–7 you should be able to:

1. state the properties of an algorithm and express an algorithm in pseudocode and as a flowchart;
2. choose suitable data types, declare variables and constants, read input and convert it (`Parse`, `TryParse`, casts);
3. evaluate C# expressions by hand, including integer division, `%`, `++`/`--`, precedence and short-circuit logic;
4. write selection logic with `if`, `else if`, `switch` and `?:`;
5. write, trace and predict counter-controlled (`for`) and condition-controlled (`while`, `do-while`) loops, including nested loops and `break`/`continue`;
6. find and fix compile-time and logic errors in short programs;
7. design and write a short, complete console program on paper.

1.2 Logistics

Item	Details
Date, room, duration	Announced on the course group (pinned message). The practice exam in Section 5 is designed for 90 minutes.
Materials	Closed book: no notes, no phones, no laptops. Bring your student ID card and a pen; a pencil and an eraser help with flowcharts.
Weight	Midterm 40 %, final 60 % of the course grade.
Scope	Weeks 1–7 (table below). Arrays and methods are *not* included; do not use them in answers.
Form	Written on paper: you write C# by hand, without a compiler or IntelliSense.

Week	Topic	Typical exam task
1	Algorithms, problem solving	properties; pseudocode; test cases; trace tables
2	Flowcharts	draw a flowchart; flowchart ↔ pseudocode
3	Variables, data types, constants	choose a type; declarations; input conversion; casts; output formatting
4	Operators	evaluate expressions; integer division; `%`; `++`; logic; short-circuit
5	`if`, `else if`, `switch`	predict output; write selection logic; validate input
6	`for` loop	accumulators; nested loops and patterns; `break`/`continue`
7	`while`, `do-while`	sentinel and validation loops; digit loops; choosing a loop

1.3 Question types and how to answer them

Type	Method
Multiple choice	Eliminate wrong options first. For code items, evaluate on scrap paper; never guess the output of an expression.
Predict the output	Keep a small table of all variables. Execute one statement at a time. Remember that C# prints `True`/`False` and that a `double` with no fractional part is printed without `.0`.
Trace table	One column per variable (and one for output), one row per pass. Write the values *after* each pass. Count condition tests separately if asked.
Flowchart	Pencil first. Standard symbols, arrows on every line, both exits of every decision labelled, exactly one Start and one End.
Find the bugs	Check every line for: misplaced `;`, `=` vs `==`, types and literals, integer division, loop bounds, missing `$`, missing checks (division by zero, empty input). State the fix, not only the error.
Write a program	Input–process–output first, then name the pattern (Section 2.4), then pseudocode, then C#. Prompts, conversions, logic, output format, in this order.

Time management

Budget roughly one minute per point and keep 8–10 minutes for checking. Read the whole paper first (5 minutes) and start with the questions you are sure about. Never leave a question blank: a correct plan, pseudocode or a partial trace usually earns partial credit.

>_ 2 Cheat sheet

This summary is for *learning*, not for the exam room (the exam is closed book). The best preparation is to write your own one-page version by hand.

2.1 C# syntax summary

Program skeleton (top-level statements)

```
// comment
const double VatRate = 0.20;
Console.Write("n = ");
int n = int.Parse(Console.ReadLine());
Console.WriteLine($"n = {n}");
```

Input and conversion

```
double x = double.Parse(Console.ReadLine());
// on failure: ok = false, v = 0
bool ok = int.TryParse(Console.ReadLine(),
                       out int v);
int t = (int)7.9;        // 7 (truncates)
double r = (double)7 / 2; // 3.5
```

Output formatting

`$"{x:F2}"`	2 decimals: `3.14`
`$"{n:N0}"`	thousands: `1,234,567`
`$"{x,8}"`	right-aligned in 8 columns
`\n \t \" \\`	escape sequences

Selection

```
if (score >= 85)
{
    letter = "BA";
}
else if (score >= 50)
{
    letter = "DD";
}
else
{
    letter = "FF";
}
switch (day)
{
    case 6:
    case 7:
        kind = "Weekend";
        break;
    default:
        kind = "Weekday";
        break;
}
string p = n % 2 == 0 ? "even" : "odd";
string g = score switch
{
    >= 85 => "BA",
    >= 50 => "DD",
    _ => "FF"
};
```

Loops

```
for (int i = 1; i <= n; i++)
{
    sum += i;
}
while (x != -1)     // 0 or more passes
{
    x = int.Parse(Console.ReadLine());
}
do                  // at least 1 pass
{
    ok = int.TryParse(Console.ReadLine(),
                      out n);
} while (!ok || n < 0);
break;      // leave the loop now
continue;   // skip to the next pass
```

Useful `Math` members

`Math.Sqrt(x)`, `Math.Pow(a, b)`, `Math.Abs(x)`, `Math.Max(a, b)`, `Math.Min(a, b)`, `Math.Floor(x)`, `Math.Ceiling(x)`, `Math.Round(x, 2)` (`Math.Round(2.5)` is 2: banker's rounding), `Math.PI`.

2.2 Data types and operators

Type	Size	Literal	Use for
`int`	4 B	`42`	whole numbers, $-2\,147\,483\,648 \ldots 2\,147\,483\,647$
`long`	8 B	`6227020800L`	large whole numbers (e.g. $13! = 6\,227\,020\,800$)
`double`	8 B	`72.5`	measurements, scientific values ($\approx$ 15–17 significant digits)
`float`	4 B	`2.5f`	rarely needed ($\approx$ 7 digits)
`decimal`	16 B	`19.99m`	money (exact decimal digits)
`char`	2 B	`'A'`	one character, single quotes
`string`	–	`"Hello"`	text, double quotes
`bool`	1 B	`true`	results of conditions

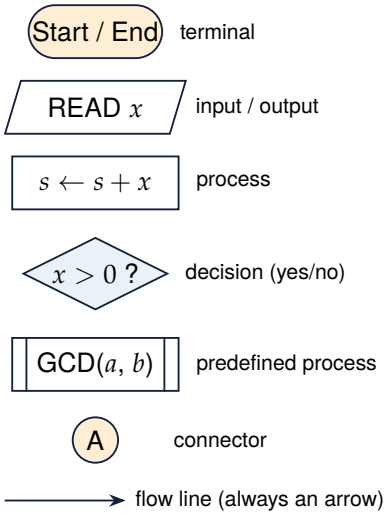
Level	Operators (high to low)	Expression	Value
1	`x++ x--`	`17 / 5`	`3`
2	`! -x ++x --x (type)x`	`17 % 5`	`2`
3	`* / %`	`-17 % 5`	`-2` (sign of the left operand)
4	`+ -`	`7 / 2.0`	`3.5`
5	`< > <= >=`	`10 - 4 / 2 * 3`	`4`
6	`== !=`	`"a" + 1 + 2`	`"a12"`
7	`&&`	`1 + 2 + "a"`	`"3a"`
8	`||`	`b != 0 && a / b > 2`	`false` if `b` is 0 (no division)
9	`?:`	`int y = x++ * 2;`	uses the old `x`, then increments
10	`= += -= *= /= %=`	`int y = ++x * 2;`	increments first, then uses `x`

Binary operators of equal precedence are evaluated left to right; assignment operators right to left. Integer division by zero throws `DivideByZeroException`; `double` division by zero gives ∞ or `NaN`.

2.3 Pseudocode conventions and flowchart symbols

Pseudocode	C#
START ... **END**	(whole program)
READ x	`x = int.Parse(...)`
PRINT x	`Console.WriteLine(x)`
$x \leftarrow x + 1$	`x = x + 1;`
$a = b$, $a \neq b$	`a == b`, `a != b`
a mod b	`a % b`
IF c **THEN** ... **ELSE** ... **END IF**	`if (c) {} else {}`
WHILE c ... **END WHILE**	`while (c) {}`
FOR $i \leftarrow 1$ **TO** n ... **END FOR**	`for (...) {}`
AND , **OR** , **NOT**	`&&`, `||`, `!`

$\leftarrow$ *assigns*, $=$ *compares; in C#* `=` *assigns and* `==` *compares.*



2.4 Loop patterns

Pattern	Skeleton	Remember
Accumulator	`sum = 0; ... sum += x;`	initialise *before* the loop (`product = 1`)
Counter	`count = 0; ... if (cond) count++;`	counts only the items that satisfy the condition
Best so far	`max = first; ... if (x > max) max = x;`	start with the first value, not with 0
Sentinel	read; `while (x != -1) { ...; read; }`	priming read before the loop, next read at the end of the body
Validation	`do { read } while (!ok || n < 0);`	repeat while the input is *bad*
Digit loop	`d = n % 10; n = n / 10;`	use `do-while` if 0 must count as one digit

>_ 3 Review questions from the lecture

These are the twelve practice questions of the Week 8 slides with worked answers. Cover the answer, solve the question on paper, then compare. All outputs shown were produced by the program `08_01_PredictChecker` in the code folder.

3.1 Q1–Q4: multiple choice

Q1. "Keep adding 1 to x." Which property of an algorithm fails? (A) definiteness (B) finiteness (C) input (D) output

Q2. Which flowchart symbol is used for **READ** x? (A) rectangle (B) diamond (C) parallelogram (D) oval

Q3. Best type for a price such as 1499.90 TL? (A) `int` (B) `float` (C) `decimal` (D) `char`

Q4. Whose body always runs at least once? (A) `for` (B) `while` (C) `do-while` (D) none

Answers

Q1 (B): there is no stopping condition. Q2 (C): input and output both use the parallelogram. Q3 (C): `decimal` stores decimal digits exactly; `float` and `double` cannot store 0.1 exactly. Q4 (C): `do-while` tests its condition after the body.

3.2 Q5: operators

```
1  int a = 17, b = 5;
2  Console.WriteLine(a / b);
3  Console.WriteLine(a % b);
4  Console.WriteLine(a / (double)b);
5  Console.WriteLine(a + b * 2 - 1);
6  Console.WriteLine(-a % b);
7  Console.WriteLine("Sum: " + a + b);
8  Console.WriteLine(a > b && b != 5);
9  int x = 5;
10 int y = x++ * 2;
11 Console.WriteLine($"{x} {y}");
```

Output

```
3
2
3.4
26
-2
Sum: 175
False
6 10
```

Explanation. $17/5 = 3$ (both operands `int`); $17 = 3 \cdot 5 + 2$; one `double` operand gives 3.4; `*` before `+`/`-`: $17 + 10 - 1 = 26$; the remainder keeps the sign of the left operand: $-17 \,\%\, 5 = -2$; `"Sum: "`

`+ 17` is already a string, so `+ 5` appends "5"; $17 > 5$ is true but $5 \neq 5$ is false, and C# prints `False`; `x++` gives the old value 5 to the expression, so $y = 10$ and then $x = 6$.

3.3 Q6: trace table

```
int n = 1230, rev = 0, count = 0;
while (n > 0)
{
    int d = n % 10;
    rev = rev * 10 + d;
    n = n / 10;
    count++;
}
Console.WriteLine($"{rev} {count}");
```

pass	d	rev	n	$count$
start	–	0	1230	0
1	0	0	123	1
2	3	3	12	2
3	2	32	1	3
4	1	321	0	4

Output: `321 4`

The condition `n > 0` is tested 5 times (4 times true, once false). The trailing zero of 1230 would become a leading zero, so the reversed number is 321.

3.4 Q7: selection

(a)

```
int score = 85;
if (score >= 50)
{
    Console.WriteLine("DD");
}
else if (score >= 85)
{
    Console.WriteLine("BA");
}
else
{
    Console.WriteLine("FF");
}
```

(b)

```
int day = 7;
switch (day)
{
    case 6:
    case 7:
        Console.WriteLine("Weekend");
        break;
    default:
        Console.WriteLine("Weekday");
        break;
}
```

(c) `Console.WriteLine(score % 2 == 0 ? "even" : "odd");` (same `score`)

Answers. (a) `DD`: only the *first* true branch of an else-if ladder runs, and $85 \geq 50$. The `"BA"` branch can never run, for any score, because every score ≥ 85 is also ≥ 50; test the most restrictive condition first. (b) `Weekend`: the grouped labels 6 and 7 share one body. (c) `odd`, since $85 \,\%\, 2 = 1$.

3.5 Q8: nested loops

```
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= 4 - i; j++)
    {
        Console.Write(" ");
    }
    for (int k = 1; k <= 2 * i - 1; k++)
    {
        Console.Write("*");
    }
    Console.WriteLine();
}
```

Output

```
␣␣␣*
␣␣***
␣*****
*******
```

Spaces shown as ␣. Row i has $4-i$ spaces and $2i-1$ stars.

`Console.Write("*")` runs $1+3+5+7=16=4^2$ times; the inner space loop runs $3+2+1+0=6$ times.

3.6 Q9: `break`, `continue`, `do-while`

(a)

```
int total = 0;
for (int i = 1; i <= 10; i++)
{
    if (i % 3 == 0)
    {
        continue;
    }
    if (i > 7)
    {
        break;
    }
    total += i;
}
Console.WriteLine(total);
```

(b)

```
int k = 10;
do
{
    Console.Write(k + " ");
    k += 5;
} while (k < 10);
Console.WriteLine($"| k = {k}");
```

Output of (a) and (b)

```
19
10 | k = 15
```

(a) $i=1,2$ are added; 3 is skipped; 4, 5 added; 6 skipped; 7 added; at $i=8$ the `break` ends the loop: $1+2+4+5+7=19$. (b) The body runs once before the test $15<10$ fails. With a `while` loop the body would not run at all and the output would be `| k = 10`.

3.7 Q10: find the bugs

The following program should read n and then n integer grades, and print their average with two decimals.

```
Console.Write("How many grades? ");
int n = int.Parse(Console.ReadLine());
int sum = 0;
for (int i = 1; i < n; i++);
{
    Console.Write("Grade: ");
    int g = int.Parse(Console.ReadLine());
    sum += g;
}
```

```
10 double avg = sum / n;
11 Console.WriteLine("Average: {avg:F2}");
```

It compiles with one warning (CS0642, "Possible mistaken empty statement", line 4). A real run:

Buggy version

```
How many grades? 3
Grade: 70
Average: {avg:F2}
```

Line	Problem	Fix
4	`i < n` runs $n-1$ times (off by one)	`i <= n`
4	`;` after the header: the loop body is empty; the block below runs once	delete the `;`
10	`sum / n` is integer division	`(double)sum / n`
11	no `$`: the text `{avg:F2}` is printed literally	`$"Average: {avg:F2}"`
2	$n = 0$ causes `DivideByZeroException`	check $n > 0$ before the loop

Listing 1: Corrected program (`08_02_AverageFixed`)

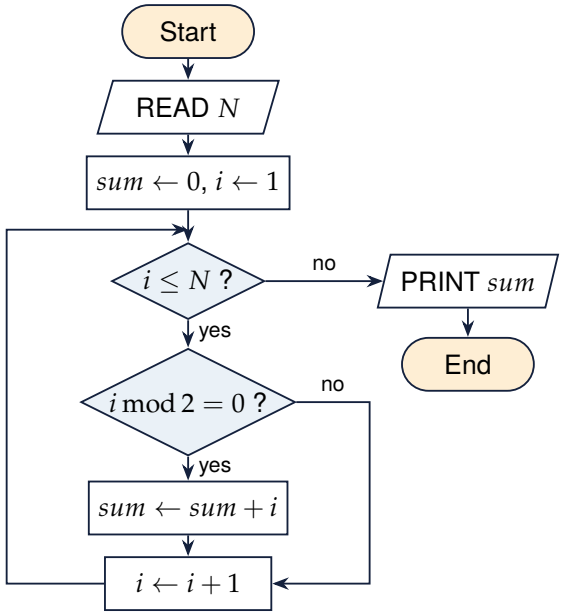
```
1  // MCMP113 · Week 8 · Example 2 --- Average of N grades (review question Q10, bugs fixed)
2  // Corrected "find the bugs" program: <= instead of <, no ';' after for, real division, $
       string, guard for n <= 0.
3
4  Console.Write("How many grades? ");
5  int n = int.Parse(Console.ReadLine());
6
7  if (n <= 0)
8  {
9      Console.WriteLine("Nothing to average.");   // fix 5: avoid division by zero
10 }
11 else
12 {
13     int sum = 0;
14     for (int i = 1; i <= n; i++)                // fix 1: <=   fix 2: no ';' here
15     {
16         Console.Write($"Grade {i}: ");
17         int g = int.Parse(Console.ReadLine());
18         sum += g;
19     }
20     double avg = (double)sum / n;               // fix 3: real (not integer) division
21     Console.WriteLine($"Average: {avg:F2}");    // fix 4: $ makes it an interpolated
       string
22 }
```

Output

```
How many grades? 3
Grade 1: 70
Grade 2: 85
Grade 3: 92
Average: 82.33
```

3.8 Q11: draw the flowchart

Read a whole number N; compute and print the sum of all even numbers from 1 to N. For $N = 10$: $2 + 4 + 6 + 8 + 10 = 30$.



Pseudocode

```
START
READ N
sum ← 0; i ← 1
WHILE i ≤ N
    IF i mod 2 = 0 THEN
        sum ← sum + i
    END IF
    i ← i + 1
END WHILE
PRINT sum
END
```

The back-arrow joins the flow line *above* the loop test. A version that starts at $i = 2$ and adds 2 each time (no inner decision) is also correct.

Listing 2: `08_03_SumOfEvens`

```
// MCMP113 · Week 8 · Example 3 --- Sum of the even numbers from 1 to N (review question
    Q11)
// C# version of the flowchart answer: a counter loop with a decision inside.

Console.Write("N = ");
int n = int.Parse(Console.ReadLine());

int sum = 0;
for (int i = 1; i <= n; i++)
{
    if (i % 2 == 0)          // even?
    {
        sum += i;
    }
}
Console.WriteLine(
    $"Sum of even numbers from 1 to {n}: {sum}");
```

Output

```
N = 10
Sum of even numbers from 1 to 10: 30
```

Other tests: $N = 7 \to 12$; $N = 0 \to 0$ (the loop body never runs).

3.9 Q12: write the program

Read integer grades until the user types −1. A grade outside 0–100 prints "Invalid grade, ignored." and is not counted. At the end print the count, the average (2 decimals), the highest grade and the number of passing grades (≥ 50). If no valid grade was entered, print "No grades entered." Use no arrays.

Design. Patterns: sentinel (priming read, next read at the end of the body) + counter + accumulator + best so far. The condition `count == 1` starts the maximum with the first valid grade.

Listing 3: `08_04_GradeStats`

```
// MCMP113 · Week 8 · Example 4 --- Grade statistics with a sentinel (review question Q12)

// Sentinel-controlled while loop: count, average, highest and number passed; invalid grades are skipped.

int count = 0, sum = 0, highest = 0, passed = 0;

Console.Write("Grade (-1 to stop): ");
int grade = int.Parse(Console.ReadLine());      // priming read

while (grade != -1)
{
    if (grade < 0 || grade > 100)
    {
        Console.WriteLine("  Invalid grade, ignored.");
    }
    else
    {
        count++;
        sum += grade;
        if (count == 1 || grade > highest)       // first valid grade starts the maximum
        {
            highest = grade;
        }
        if (grade >= 50)
        {
            passed++;
        }
    }
    Console.Write("Grade (-1 to stop): ");
    grade = int.Parse(Console.ReadLine());      // read the next value at the END of the body
}

if (count == 0)
{
    Console.WriteLine("No grades entered.");
}
else
{
    double average = (double)sum / count;
    Console.WriteLine($"Count: {count}");
    Console.WriteLine($"Average: {average:F2}");
    Console.WriteLine($"Highest: {highest}");
    Console.WriteLine($"Passed: {passed} of {count}");
}
```

Output

```
Grade (-1 to stop): 72
Grade (-1 to stop): 45
Grade (-1 to stop): 105
Invalid grade, ignored.
Grade (-1 to stop): 90
Grade (-1 to stop): 58
Grade (-1 to stop): -1
Count: 4
Average: 66.25
Highest: 90
Passed: 3 of 4
```

Test input	Output (count, average, highest, passed)
`72 45 105 90 58 -1`	4, 66.25, 90, 3 (105 rejected)
`-1`	No grades entered.
`30 40 -1`	2, 35.00, 40, 0
`50 -1`	1, 50.00, 50, 1 (boundary: 50 passes)

>_ 4 Common mistakes

Ten mistakes that cost points

1. `if (x = 5)` instead of `if (x == 5)`: for an `int x` this does not compile (CS0029, cannot convert `int` to `bool`).
2. `int n = Console.ReadLine();`: `ReadLine` returns a `string`; use `int.Parse` or `int.TryParse`.
3. `decimal p = 9.99;`: a literal without suffix is a `double` (CS0664); write `9.99m`.
4. Integer division by zero crashes (`DivideByZeroException`); check the divisor first.
5. `(a + b) / 2` with `int` operands is integer division; use `2.0` or a cast.
6. `0 <= x <= 100` is not valid C#; write `x >= 0 && x <= 100`.
7. A `;` directly after `for (...)` or `while (...)` makes the loop body empty.
8. Off-by-one: `i < n` vs `i <= n`; always check the first and the last pass.
9. A `while` loop whose variable is never updated never ends.
10. Starting a maximum at 0 fails when all values are negative; start with the first value.

Mistakes specific to hand-written exams

Unreadable handwriting in code; missing braces that change the meaning; answering a different question (read the specification twice); printing extra text in a "predict the output" answer; forgetting the prompts (`Console.Write`) before reading input; leaving a question blank.

>_ 5 Practice Midterm Exam

MCMP113 · Practice Midterm Exam Duration: 90 minutes · Total: 100 points

Name: _____ Student no.: _____

Instructions. Closed book: no notes, phones or calculators. Answer on the question paper where space is given and on separate answer sheets otherwise. Write C# with top-level statements; do not use arrays or methods. Show your work: partial credit is given for correct reasoning, pseudocode and partial traces.

Question	1	2	3	4	5	6	7	Total
Topic	MC	Predict	Trace	Flowchart	Bugs	Selection	Loops	
Points	20	15	15	12	13	12	13	100
Score								

Question 1 – Multiple choice [20 pts]

Circle one answer per item (2 points each).

1.1 Which of the following is *not* one of the five properties of an algorithm?
(a) Finiteness (b) Definiteness (c) Effectiveness (d) Portability

1.2 In a flowchart, which symbol must have exactly two labelled exits?
(a) Terminal (b) Process (c) Decision (d) Input/output

1.3 Which one is a valid C# identifier?
(a) `2ndGrade` (b) `total-sum` (c) `_count` (d) `class`

1.4 What is the value of `x` after `int x = (int)7.9;`?
(a) `8` (b) `7` (c) `7.9` (d) compile error

1.5 What is the value of `10 - 4 / 2 * 3`?
(a) `9` (b) `4` (c) `10` (d) `12`

1.6 Which expression is true exactly when `x` is between 1 and 10 inclusive?
(a) `1 <= x <= 10` (b) `x >= 1 || x <= 10` (c) `x >= 1 && x <= 10` (d) `x => 1 && x =< 10`

1.7 What does `Console.WriteLine(7 / 2 * 2.0);` print?
(a) `7` (b) `6` (c) `7.0` (d) `3.5`

1.8 How many times is `Hi` printed by the following loop?

```
for (int i = 0; i < 10; i += 3) { Console.WriteLine("Hi"); }
```

(a) 3 (b) 4 (c) 10 (d) infinitely many

1.9 Which statement about the `do-while` loop is true?
(a) its body may execute zero times (b) the condition is checked before the body
(c) its body executes at least once (d) it cannot contain `break`

1.10 After `bool ok = int.TryParse("12a", out int v);` the values are:
(a) `ok = true, v = 12` (b) `ok = false, v = 0` (c) `ok = false, v = 12`
(d) a `FormatException` is thrown

Question 2 – Predict the output [15 pts]

Write the exact output of each fragment (3 points each). Each fragment runs on its own.

(a)

```
int a = 7, b = 2;
double c = a / b;
Console.WriteLine($"{c} {a % b} {(double)a / b
    }");
```

(b)

```
string s = "C#";
Console.WriteLine(s + 1 + 2);
Console.WriteLine(1 + 2 + s);
```

(c)

```
int x = 5;
int y = x++ + 2;
int z = --x * 2;
Console.WriteLine($"{x} {y} {z}");
```

(d)

```
int t = 25;
bool raining = false;
if (t > 20 && !raining)
{
    Console.Write("Picnic ");
}
if (t > 10 || raining)
{
    Console.Write("Walk ");
}
else
{
    Console.Write("Stay ");
}
Console.WriteLine(t >= 25 ? "Warm" : "Cool");
```

(e)

```
int month = 4, days;
switch (month)
{
    case 2:
        days = 28;
        break;
    case 4:
    case 6:
    case 9:
    case 11:
        days = 30;
        break;
    default:
        days = 31;
        break;
}
Console.WriteLine($"Month {month}: {days} days");
```

Question 3 – Trace table [15 pts]

```
int n = 13, steps = 0, maxValue = n;
while (n != 1)
{
    if (n % 2 == 0)
    {
        n = n / 2;
    }
    else
    {
        n = 3 * n + 1;
    }
    steps++;
    if (n > maxValue)
    {
        maxValue = n;
    }
}
Console.WriteLine($"steps={steps}, max={maxValue}"
    );
```

(a) Complete the trace table: one row per pass, values at the *end* of the pass. [10 pts]

(b) Write the output. [3 pts]

(c) How many times is the condition `n != 1` evaluated? [2 pts]

pass	n	*steps*	*maxValue*
start	13	0	13
1			
2			
…			
(add rows as needed)			

Question 4 – Flowchart [12 pts]

Draw a flowchart for the following algorithm. The program reads integers one by one until the user enters 0. It then prints how many of the entered numbers were positive and how many were negative (the final 0 is not counted). Use the standard symbols.

Question 5 – Find and fix the bugs [13 pts]

The program below should read an integer n with $1 \le n \le 20$ and print $n!$ (n factorial); for other values it should print `Out of range`. It contains **five** errors.

```
// Intended: read n (1..20) and print n! (n factorial)
Console.Write("n = ");
int n = int.Parse(Console.ReadLine())
if (n < 1 && n > 20)
{
    Console.WriteLine("Out of range");
}
else
{
    int fact = 0;
    for (int i = 1; i <= n; i++);
    {
        fact = fact * i;
    }
    Console.WriteLine("n! = " + fact);
}
```

(a) For each error give the line number, explain the problem and write the corrected line. [10 pts]

(b) Which of the errors does the compiler report, and which ones are logic errors that only testing can reveal? [3 pts]

Question 6 – Write a program: selection [12 pts]

A cargo company computes shipping costs as follows. Write a C# program that reads the **weight** in kg (a decimal number) and the **zone** (an integer) and prints the base price, the surcharge and the total with two decimals.

- Use `TryParse`. If the weight is not a number, ≤ 0 or > 30, print `Invalid weight.` If the zone is not 1, 2 or 3, print `Invalid zone.` In both cases compute nothing.
- Base price by zone (use a `switch`): 1 (same city) 60 TL, 2 (domestic) 90 TL, 3 (international) 350 TL.
- Weight surcharge: up to 2 kg none; above 2 kg up to 10 kg, 8 TL for every kg above 2; above 10 kg, 64 TL plus 12 TL for every kg above 10.

Example: 5 kg to zone 1 costs $60 + 3 \cdot 8 = 84$ TL.

Question 7 – Write a program: loops [13 pts]

Write a C# program that

(a) reads an integer N and repeats the input (with the message `Invalid input, try again.`) until a whole number $N \geq 2$ is entered; use a `do-while` loop and `TryParse`; [3 pts]

(b) prints all divisors of N on one line; [4 pts]

(c) prints the number of divisors; [2 pts]

(d) prints whether N is prime (a prime has exactly two divisors); [2 pts]

(e) prints whether N is *perfect*: the sum of its divisors smaller than N equals N (e.g. $6 = 1 + 2 + 3$). [2 pts]

— End of the practice exam —

>_ 6 Additional problem set

Difficulty: ⋆ basic, ⋆⋆ intermediate, ⋆⋆⋆ challenging. For programming problems, write the pseudocode first; the tested solutions are in the code folder (projects `08_08`–`08_14`).

P1. ⋆ **Counting passes.** How many times is `count++` executed in each fragment?
(a) `for (int i = 1; i <= 5; i++) { for (int j = i; j <= 5; j++) { count++; } }`
(b) `for (int i = 1; i <= 5; i++) { for (int j = 1; j < i; j++) { count++; } }`

P2. ⋆ **Trace.** Trace the loop and give the output.

```
int x = 1;
for (int i = 1; i <= 5; i++)
{
    x = x * 2 + i;
}
Console.WriteLine(x);
```

P3. ⋆ **Convert.** Rewrite `for (int i = 10; i > 0; i -= 3) { Console.Write(i + " "); }` as a `while` loop and as a `do-while` loop. What does each version print? Do the three versions still behave the same if the start value is 0 instead of 10?

P4. ⋆⋆ **Minimum, maximum, average.** Read $N \geq 1$ and then N real numbers. Print the smallest, the largest and the average (2 decimals). Draw the flowchart first. Do not use arrays.

P5. ⋆⋆ **Days in a month.** Read a year and a month number (validate both with `TryParse`; month 1–12). Print the number of days in that month, taking leap years into account, and whether the year is a leap year. Use a `switch` with grouped `case` labels.

P6. ⋆⋆ **Hollow square.** Read a size n ($2 \leq n \leq 20$, repeat the input until valid) and print a hollow square of * characters, each followed by a space.

P7. ⋆⋆ **Digit statistics.** Read an integer $0 \leq n \leq 999\,999\,999$. Print the number of digits, the digit sum, the number of even digits, the reversed number and whether n is a palindrome. Why is the upper limit needed, and why is a `do-while` loop better than a `while` loop here?

P8. ⋆⋆ **Menu calculator.** Write a menu-driven calculator (`do-while` + `switch`): 1 add, 2 subtract, 3 multiply, 4 divide, 0 exit. Reject division by zero and unknown choices. What happens if the user types a letter as the choice and you do not handle it?

P9. ⋆⋆⋆ **Primes in a range.** Read A and B with $2 \leq A \leq B$ (repeat until valid). Print all primes between A and B and how many there are. Test divisors only up to $\sqrt{n}$ and stop at the first divisor.

P10. ⋆⋆⋆ **Savings goal.** Every month you deposit a fixed amount into an account that pays a fixed monthly interest rate (applied to the balance before the new deposit). Read the deposit, the rate in percent and the target, and print after how many months the target is reached and the final balance. Use `decimal`.

P11. ⋆ **The loop that never ends.** What does this fragment print, and why? Fix it so that it prints `1 3 5 7 9`.

```
int i = 1;
while (i != 10)
{
    Console.Write(i + " ");
    i += 2;
}
```

P12. ⋆⋆ **Logic and short-circuit.** With `int a = 3, b = 0, c = -2;` evaluate:
(a) `a > b && b > c` (b) `!(a < b) || c > 0` (c) `b != 0 && a / b > 1`
(d) `a % 2 == 1 || a / b > 0` (e) `!(a > 0 && c > 0)` (f) `a / b > 0 || a % 2 == 1`

>_ 7 Summary

1. The midterm covers Weeks 1–7: algorithms, flowcharts, data types, operators, selection and loops; no arrays, no methods.
2. Predict and trace with a table: one column per variable, one row per step or pass.
3. Watch integer division, the sign of `%`, precedence, `++` prefix/postfix and `string + number`.
4. In an else-if ladder only the first true branch runs; order the conditions from the most restrictive.
5. Choose the loop: `for` when the number of passes is known, `while` for conditions and sentinels, `do-while` when the body must run at least once.
6. Most programming questions combine a few patterns: accumulator, counter, best so far, sentinel, validation, digit loop.

7. Plan before you write: IPO, pattern, pseudocode, then C#; test normal, boundary and invalid inputs.

Glossary (English – Turkish)

midterm exam	ara sınav	closed-book	kitap/not kullanılmadan
predict the output	çıktıyı tahmin etmek	trace table	izleme tablosu
syntax error	sözdizimi hatası	compile-time error	derleme zamanı hatası
run-time error	çalışma zamanı hatası	logic error	mantık hatası
off-by-one error	bir fazla/eksik hatası	warning	uyarı
sentinel value	gözcü (bitiş) değeri	input validation	girdi doğrulama
accumulator	biriktirici (toplam) değişken	counter	sayaç
infinite loop	sonsuz döngü	nested loop	iç içe döngü
short-circuit evaluation	kısa devre değerlendirme	precedence	(işlem) önceliği
integer division	tam sayı bölmesi	remainder	kalan
divisor	bölen	prime number	asal sayı
perfect number	mükemmel sayı	palindrome	palindrom (tersten de aynı)
leap year	artık yıl	rubric	puanlama anahtarı
partial credit	kısmi puan	overflow	taşma

References and further reading

[1] Microsoft, *Selection statements: if, if-else and switch*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/selection-statements

[2] Microsoft, *Iteration statements: for, foreach, do and while*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/iteration-statements

[3] Microsoft, *Jump statements: break, continue, return and goto*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/statements/jump-statements

[4] Microsoft, *C# operators and expressions* (precedence table). https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/

[5] Microsoft, *Arithmetic operators*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/arithmetic-operators

[6] Microsoft, *Boolean logical operators*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/boolean-logical-operators

[7] Microsoft, *switch expression*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/switch-expression

[8] Microsoft, *Built-in types*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/built-in-types

[9] Microsoft, *Int32.TryParse method*. https://learn.microsoft.com/en-us/dotnet/api/system.int32.tryparse

[10] Microsoft, *Standard numeric format strings*. https://learn.microsoft.com/en-us/dotnet/standard/base-types/standard-numeric-format-strings

[11] MCMP113 Week 1–7 lecture notes and code folders.